Learning When to Ask

Trust Calibration for Agentic Tool Use,
from Known Tools to Opaque LLM Judges

Changkun Ou
Latere AI, Munich, Germany*

September 28, 2026

Abstract

We formalize trust calibration for agentic tool use (deciding whether an agent's proposed action may run autonomously, must be escalated to a human, or is blocked) as Gaussian-process classification with a reject option, learned online from the supervisor's approve/deny feedback. The three-tier gateway is the Bayes-optimal one-step decision under Chow's reject-option loss, with thresholds $\tau_{\text{high}} = 1 - c_{\text{ask}}/c_{\text{FA}}$ and $\tau_{\text{low}} = c_{\text{ask}}/c_{\text{FB}}$ set by stated costs rather than tuned. We prove that unary approve/deny feedback identifies the allow/deny boundary while pairwise (preferential) feedback cannot; that a separable time-decaying kernel forgets static action risk and forces every unlabelled action back into escalation; that the irreducible escalation rate is a property of the supervisor; and that random audits give unbiased false-allow estimates, with $n \geq \ln(1/\delta)/\alpha$ clean audits certifying a rate below α. In simulation, an additive kernel that separates static risk from drifting tolerance halves human labels per action (28.4% $\to$ 13.5%) and cuts cost-weighted regret by 59% at an unchanged false-allow rate. We then replace the tool taxonomy with an opaque LLM judge, as in shell-centric agent harnesses: a judge's error rate for a supervisor is not identifiable from its verdicts, changes when the supervisor's tolerance changes, and is estimable without bias only from labels with known propensities. In a pre-registered simulation the calibrated gateway matches the regret of escalating the judge's blocks with half the human labels, but cannot repair a judge blind spot its features do not expose. The cores of all eight propositions are machine-checked in Lean 4 with Mathlib. The problem is related to, but not an instance of, preferential Bayesian optimization.

1 Introduction

Coding agents today gate tool calls with a binary switch: run automatically, or refuse. A refusal does not pause the run for a human; the agent receives it and keeps going, often routing around the obstacle in ways that are worse than asking would have been. The missing primitive is not *block* but *escalate*: an action the system is unsure about should go to the supervisor, and the point at which to escalate should be learned from the supervisor's own approve/deny history rather than written by hand. A supervisor who has approved the same test command a hundred times should not be asked again; a supervisor who has just moved to an unfamiliar production system should be asked more often, even about actions approved yesterday.

Two developments make this learning problem both more pressing and harder. First, agent harnesses increasingly route work through a general-purpose shell, so a proposed action is arbitrary

*Code, simulations and machine-checked proofs: https://github.com/changkun/trustcalib.

	Question	Theory	Experiment
RQ1	Decision rule and thresholds	Props. 2–3 (§3.4)	§6.1
RQ2	Feedback that identifies the boundary	Prop. 1 (§3.2), §2	–
RQ3	What to forget under drift	Prop. 4 (§3.5–3.6)	§6
RQ4	Error rates under selective labels	Prop. 5 (§3.7)	§7
RQ5	Trusting an opaque judge	Props. 6–8 (§4)	§8

Table 1: Research questions and where the paper answers them.

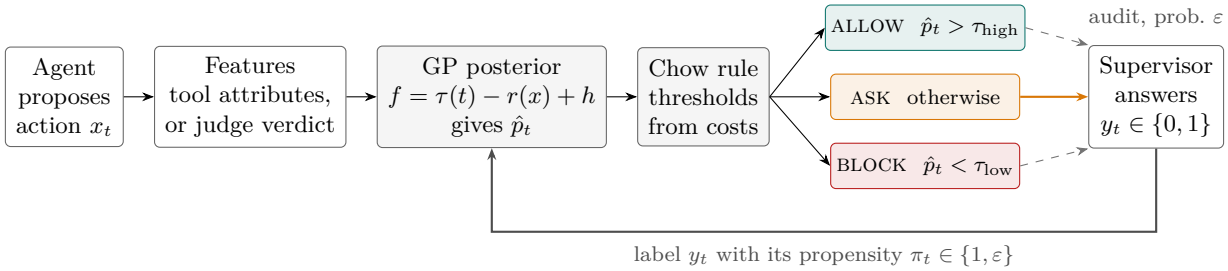


Figure 1: The escalation gateway. Each proposed action is mapped to observable features: the attributes of a known tool, or, in a shell-centric harness, an opaque judge's verdict and a command category. A GP posterior over the supervisor's latent tolerance gives the approval probability $\hat{p}_t$, and Chow's reject-option rule turns it into ALLOW, ASK or BLOCK. Only escalated actions and a random ε fraction of auto-decided ones (audits) are shown to the supervisor; their answers, each with its sampling propensity, are the only training labels.

code rather than a call to one of a known set of tools. Second, the allow/block decision is increasingly delegated to a second language model, a *judge* that reads the command and part of the conversation and returns a verdict [28, 39]. The judge is opaque to the supervisor, and it raises two questions that a fixed permission list never did: is the judge trustworthy, and when it says allow or block, how likely is that verdict to be what the supervisor would have decided?

This paper formalizes the escalation decision as online Gaussian-process (GP) classification with a reject option, learned from the supervisor's approve/deny answers, and asks five research questions:

RQ1 *Decision.* Which decision rule should the gateway apply to its posterior, and where should its thresholds come from?

RQ2 *Feedback.* Which kind of supervisor feedback identifies the boundary between actions to allow and actions to deny?

RQ3 *Memory.* What should a model of a drifting supervisor forget, and what does forgetting cost?

RQ4 *Evaluation.* How can the gateway know its own error rate when the actions it decides alone are never labelled?

RQ5 *Opaque judges.* When an LLM judge replaces the tool taxonomy, is the judge trustworthy, and how likely is a given verdict to be right?

Table 1 shows where each question is answered, and Figure 1 shows the gateway that the answers assemble.

Contributions. The paper makes six contributions.

- *Decision rule (**RQ1**).* The three-tier ALLOW/ASK/BLOCK gateway is exactly Chow's reject-option rule [12]: its thresholds follow in closed form from the costs of a false allow, a false block and an escalation, and it depends on the posterior only through the mean approval probability (Proposition 2). Even a perfect posterior escalates every action on which the supervisor is ambivalent at those costs, so the achievable automation rate is a property of the supervisor (Proposition 3).
- *Identifiability (**RQ2**).* Unary approve/deny feedback identifies the allow/deny boundary; pairwise feedback cannot, because it is invariant to shifting the latent function by a constant (Proposition 1). This places the problem next to, not inside, preferential Bayesian optimization.
- *Forgetting (**RQ3**).* A separable time-decaying kernel forgets static risk and forces every unlabelled action back into escalation after at most $\lambda \ln(m/z)$ label-free steps (Proposition 4). An additive decomposition of the latent tolerance into static risk and drifting tolerance forgets only what drifts.
- *Selective labels (**RQ4**).* Auto-decided actions are never labelled, so every error rate computed from the gateway's labels is biased. Random audits restore unbiased estimates, and certifying a false-allow rate below α at confidence $1 - \delta$ takes $\ln(1/\delta)/\alpha$ clean audits (Proposition 5).
- *Opaque judges (**RQ5**).* A judge's error rate for a supervisor is not identifiable from its verdicts, rises when the supervisor becomes stricter, and is estimable only from human labels with known propensities (Propositions 6–8). Treating the judge's output as a feature of the gateway answers the per-verdict question with a calibrated posterior.
- *Evidence.* Simulation studies with a known oracle test each claim, one of them pre-registered; the cores of all eight propositions are machine-checked in Lean 4 with Mathlib; the simulation code and the proofs are released.

Outline. Section 2 reviews related work. Section 3 develops the gateway and its theory, and Section 4 extends it to an opaque judge. Section 5 describes the simulation environment shared by three experiments: learning a drifting supervisor (Section 6), auditing auto-decisions (Section 7) and calibrating a simulated judge (Section 8). Section 9 answers the research questions, draws implications for agent harnesses and states the limitations. Proofs are in Appendix A, the correspondence to the machine-checked theorems in Appendix B, and simulation details in Appendix C.

2 Related Work

Selective classification and deferral. A classifier that may abstain at a cost is Chow's reject option [12], developed as selective classification [20, 22]; learning when to hand a decision to a human expert is learning to defer [35, 38]. Our ASK tier is the reject option, with the twist that the human's answer on an abstained action is also the only source of training labels, the setting of online selective classification with limited feedback [21].

Gaussian-process classification and preference learning. The inference is classical GP classification: a GP prior, a probit likelihood and an approximate posterior by the Laplace method or expectation propagation [36, 42]. Chu and Ghahramani [13] introduced GP preference learning with a pairwise probit likelihood, and González et al. [24] embedded preference learning in the

query loop of Bayesian optimization [9, 47], placing a GP on the space of duels; Benavoli and Azzimonti [5] survey GP preference and choice models.

Active learning and level-set estimation. Choosing which actions to send to the human is stream-based selective sampling [14, 46]: the agent, not the learner, chooses the action, and the learner only decides whether to ask. Learning the region where f exceeds a threshold is level-set estimation [8, 25], studied with Bernoulli outcomes by Letham et al. [34]; information-theoretic acquisition for GP classification is BALD [27], and uncertainty sampling is known to suffer from sampling bias [15, 16].

Non-stationarity. A supervisor's tolerance drifts. Time-varying GP bandits model drift with a forgetting kernel multiplied onto the spatial one [7], of which our separable kernel is the continuous-time analogue; non-stationary covariances [40] and online changepoint detection [1] are alternatives. Additive kernels [19] let a model forget some components and not others.

LLM judges and monitors. Language-model judges now gate agent actions in deployed coding agents [28, 39]. Calibrated selective evaluation with an agreement guarantee [29], prediction-powered and active inference from few human labels [4, 52], latent-class models of annotator error [2, 17], and AI control with trusted monitors and audit budgets [6, 26] all bear on how far such a judge can be trusted; judges' errors are correlated across models [23, 30] and they can be attacked [43].

Trust in automation and agent governance. Appropriate reliance aligns trust with trustworthiness [33], and trust calibration is longitudinal [18]. Graduated autonomy is an explicit deployment axis for language-model agents [37]; recent work catalogues their risks [10], argues for visibility and oversight [11] and for human approval of consequential actions [48]. These accounts leave the escalation policy as a fixed tier; our contribution is a learning rule for it. Because auto-decided actions produce no label, the evaluation of such a rule faces the selective-labels problem [32].

Relation to preferential Bayesian optimization. Table 2 places the gateway next to preferential Bayesian optimization (PBO). The two share a latent function observed through a binary-response likelihood and the same approximate inference. They differ in three structural ways. First, in PBO the learner chooses the query; here the agent's stream chooses the action and the learner only chooses whether to ask (stream-based selective sampling). Second, PBO seeks an optimizer, while the gateway seeks a level set, $\{x : f(x) > \Phi^{-1}(\tau)\}$. Third, and most fundamentally, pairwise data identify f only up to a constant (Proposition 1), which suffices to find a maximizer but not a threshold. The unary observation model is therefore not a degenerate pairwise comparison against a fixed reference point: the reference point is precisely what pairwise data cannot supply. We regard the gateway as GP classification with a reject option and a level-set objective, a relative of PBO rather than an instance of it. (In González et al. [24] the GP is placed on the duel space $\mathcal{X} \times \mathcal{X}$ with a logistic link; the difference form $\Phi(f(x_i) - f(x_j))$ is that of Chu and Ghahramani [13].)

Component	PBO	Trust calibration (this paper)
Input space	design parameters	(action, context, time)
Who picks the query	learner	agent; learner only decides to ask
Feedback	pairwise preference	unary approve/deny
Identified	f up to a constant	f (Prop. 1)
Objective	$\arg\max f$	level set $\{\hat{p} > \tau_{\text{high}}\}$ etc.
Decision rule	acquisition function	Chow rule (Prop. 2)
Unlabelled decisions	none	ALLOW/BLOCK (selective labels)

Table 2: Structural comparison with preferential Bayesian optimization.

3 Trust Calibration as Classification with a Reject Option

This section builds the gateway of Figure 1 for an agent with a known tool set: the decision loop, the latent tolerance it learns, the GP model, the decision rule, and three structural properties (forgetting, correlated generalization and selective labels) that shape what it can achieve.

3.1 The escalation loop

At each decision point $t = 1, 2, \ldots$, the gateway observes a proposed agent action $a_t \in \mathcal{A}$ and an execution context $c_t \in \mathcal{C}$,

$$a_t = (\texttt{tool_name}, \texttt{args}, \texttt{target_resource}), \tag{1}$$

$$c_t = (\texttt{repo_state}, \texttt{task_desc}, \texttt{session_history}), \tag{2}$$

and writes $x_t := (a_t, c_t) \in \mathcal{X}$. The agent chooses x_t; the gateway chooses only a decision $d_t \in \{\text{ALLOW}, \text{ASK}, \text{BLOCK}\}$. If $d_t = \text{ASK}$ the human supervisor returns $y_t \in \{0, 1\}$ (deny/approve), which both settles the action and becomes a training label; ALLOW and BLOCK are executed without a label. Decisions are made prequentially: d_t depends only on labels received before t.

3.2 Latent tolerance and identifiability

Definition 1 (Latent tolerance)**.** *There is a latent function $f\colon \mathcal{X} \times \mathbb{R} \to \mathbb{R}$ such that the supervisor approves x at time t with probability*

$$\Pr(y = 1 \mid x, t) = \Phi\big(f(x, t)\big), \qquad f(x, t) = \tau(t) - r(x) + h(x, t), \tag{3}$$

where Φ is the standard normal CDF, $r(x)$ is the static risk of the action, $\tau(t)$ is the supervisor's current tolerance, and h is a (typically small) interaction that lets tolerance drift differently for different actions.

The decomposition separates what is a property of the action from what is a property of the supervisor: r can be shared across supervisors (and is what a static corpus such as R-Judge [50] could inform), while τ is one-dimensional, per supervisor, and drifts. The split is identified only up to a shared constant, which the prior fixes. The first question is which feedback identifies what the gateway needs, the sign of f.

Proposition 1 (Unary feedback identifies the boundary; pairwise does not)**.** *Let Φ be strictly increasing. (a) If $\Phi(f(x)) = \Phi(g(x))$ for all x, then $f = g$; in particular the allow region $\{x : f(x) > 0\}$ is identified by unary approval probabilities. (b) The pairwise likelihood $\Pr(x \succ x') = \Phi(f(x) - f(x'))$ is invariant under $f \mapsto f + c$ for every constant c; hence for every f and every x there is a g with identical pairwise likelihoods for all pairs and the opposite sign at x.*

Part (b) says that preference comparisons, however many, cannot tell whether an action lies on the allow or the deny side: only its rank relative to other actions is observable. Unary approve/deny feedback carries exactly the absolute information that pairwise feedback lacks (**RQ2**).

3.3 Gaussian-process prior and posterior

We place a GP prior $f \sim \mathcal{GP}(0, k)$ [42]. With feature maps for the action and context, let k_{tool} and k_{ctx} be squared-exponential kernels, $k_x = k_{\text{tool}}\, k_{\text{ctx}}$, and $k_{\text{time}}(t, t') = \exp(-|t - t'|/\lambda)$. We compare two kernels:

$$k_{\text{prod}}\big((x,t),(x',t')\big) = \sigma^2\, k_x(x,x')\, k_{\text{time}}(t,t'), \tag{4}$$

$$k_{\text{add}}\big((x,t),(x',t')\big) = \sigma_s^2\, k_x(x,x') + \sigma_g^2\, k_{\text{time}}(t,t') + \sigma_d^2\, k_x(x,x')\, k_{\text{time}}(t,t'). \tag{5}$$

The product kernel (4) is the standard separable construction for a function that drifts in time. The additive kernel (5) mirrors Definition 1: the first term is a prior on the static risk r, the second on the shared tolerance τ (every label, for any action, updates it), and the third on the interaction h. Both are positive semidefinite, since sums and Schur products of PSD kernels are PSD.

After observing $\mathcal{D}_N = \{(x_i, t_i, y_i)\}_{i=1}^N$ the posterior $p(f \mid \mathcal{D}_N) \propto \mathcal{GP}(0,k) \prod_i \Phi(f_i)^{y_i}(1 - \Phi(f_i))^{1-y_i}$ is intractable; we use the Laplace approximation (42, Algorithms 3.1–3.2), whose predictive latent mean at x_* is $\mu_* = \mathbf{k}_*^\top \nabla \log p(\mathbf{y} \mid \hat{\mathbf{f}})$ and variance $\sigma_*^2 = k_{**} - \mathbf{k}_*^\top (K + W^{-1})^{-1} \mathbf{k}_*$. Under this Gaussian approximation the predictive approval probability is exactly

$$\hat{p}(x_*) \coloneqq \mathbb{E}\big[\Phi(f_*)\big] = \Phi\big(\mu_* / \sqrt{1 + \sigma_*^2}\big). \tag{6}$$

The Laplace approximation is known to underestimate the magnitude of the posterior mean and so to give over-cautious probabilities [31]; Section 6.1 shows that the kernel, not the approximation, dominates the calibration error here. Hyperparameters can be chosen by maximizing the Laplace marginal likelihood; Section 6.1 also shows why this average-case criterion should not alone choose the forgetting rate.

3.4 The decision rule

Let an erroneous ALLOW cost c_{FA}, an erroneous BLOCK cost c_{FB}, and one escalation cost c_{ask}, all positive. For an approval probability p the expected losses are $L_{\text{ALLOW}}(p) = (1-p)\, c_{\text{FA}}$, $L_{\text{BLOCK}}(p) = p\, c_{\text{FB}}$ and $L_{\text{ASK}} = c_{\text{ask}}$.

Proposition 2 (Bayes-optimal three-tier rule)**.** *Let $\tau_{high} = 1 - c_{ask}/c_{FA}$ and $\tau_{low} = c_{ask}/c_{FB}$. (a) For any posterior μ over the latent value and integrable link, the posterior expected loss of each action equals its loss at the posterior-mean approval probability $\hat{p} = \int \Phi\, d\mu$; hence the Bayes action depends on the posterior only through $\hat{p}$. (b) The band is non-empty, $\tau_{low} < \tau_{high}$, iff $c_{ask}(c_{FA} + c_{FB}) < c_{FA}\, c_{FB}$. (c) If it is, the rule*

$$d(\hat{p}) = \begin{cases} \textit{ALLOW} & \textit{if } \hat{p} > \tau_{high}, \\ \textit{BLOCK} & \textit{if } \hat{p} < \tau_{low}, \\ \textit{ASK} & \textit{otherwise} \end{cases} \tag{7}$$

attains the minimum expected loss over the three actions for every $\hat{p}$, strictly so off the two thresholds. (d) If it is not, some auto-decision is always at least as good as asking.

A symmetric band $(0.35, 0.65)$, for example, is this rule with $c_{\mathrm{FA}} = c_{\mathrm{FB}} = c_{\mathrm{ask}}/0.35$. Asymmetric costs give asymmetric bands: with $c_{\mathrm{FA}} = 10$, $c_{\mathrm{FB}} = 4$, $c_{\mathrm{ask}} = 1$ the band is $(0.25, 0.90)$. The thresholds are therefore *specified* by the operator, not tuned on data (**RQ1**). This matters because tuning them needs labels on auto-decided actions, which the gateway never obtains (Section 3.7); tuning them on a simulator's oracle labels uses information no deployment has.

Proposition 3 (Escalation floor)**.** *Let $q(x) = \Phi(f(x))$ be the true approval probability. Any policy that is pointwise Bayes-optimal given q (that is, with a perfect posterior) must escalate every action with $\tau_{low} < q(x) < \tau_{high}$. Consequently, for any finite population of actions with nonnegative weights, its escalated mass is at least the mass of the open band.*

The achievable automation rate is thus bounded by the supervisor's own ambiguity at the operator's costs, which no learner can remove; an auto-approval rate does not simply "emerge" as the posterior concentrates. In our simulation the floor is 10.3% of actions at the symmetric band but 31.3% at the safety-weighted band.

Remark 1 (Escalation versus acquisition)**.** *Proposition 2 is a one-step rule: it values the human's answer only for the action at hand. An answer also improves future decisions, so a fully sequential rule would ask when $c_{ask} - \mathrm{VOI}(x) < \min(L_{ALLOW}, L_{BLOCK})$, with* VOI *the expected reduction in future loss. The* ASK *band is therefore an escalation rule first; whether it is also a good acquisition rule is an empirical question, and Section 6.2 shows that the answer depends on the kernel.*

Remark 2 (Epistemic margin)**.** *Because $\hat{p} = \Phi(\mu/\sqrt{1+\sigma^2})$, large posterior variance already pulls $\hat{p}$ toward $\frac{1}{2}$. An operator who wants an explicit margin can instead allow only when $\Pr(\Phi(f_*) > \tau_{high}) \geq 1-\delta$, which for a Gaussian posterior and strictly increasing Φ is $\mu_* - z_{1-\delta}\,\sigma_* > \Phi^{-1}(\tau_{high})$, a lower confidence bound in the style of safe exploration [49].*

3.5 Drift and forgetting

The time kernel k_{time} is the Ornstein–Uhlenbeck covariance: for fixed x, $f(x, \cdot)$ is a mean-reverting Markov process. It lets recent labels dominate, which tracks drift, but it also reverts every unlabelled action to the prior mean.

Proposition 4 (Forgetting)**.** *Let $\mu(x_*, t) = \sum_i k\big((x_*, t), (x_i, t_i)\big)\,\alpha_i$ be any kernel predictor whose coefficients α are fixed by the training data (the Laplace GP-probit mean has $\alpha = \nabla \log p(\mathbf{y} \mid \hat{\mathbf{f}})$), and let $t \geq \max_i t_i$ and $\Delta \geq 0$. (a) Under k_{prod}, $\mu(x_*, t+\Delta) = e^{-\Delta/\lambda}\mu(x_*, t)$, and the variance reduction $\mathbf{k}_*^\top A\,\mathbf{k}_*$ (for any fixed A) decays by $e^{-2\Delta/\lambda}$. (b) Hence, if no further labels arrive, Φ is continuous at 0 with $\Phi(0) = \frac{1}{2}$ and $\tau_{low} < \frac{1}{2} < \tau_{high}$, every action is eventually escalated; explicitly, an action with $\mu(x_*, t) = m > 0$ is no longer auto-allowed after at most $\lambda \ln(m/z)$ label-free steps, where $z = \Phi^{-1}(\tau_{high}) > 0$. (c) Under k_{add}, only the time-coupled components decay: $\mu(x_*, t+\Delta) \to \sum_i \sigma_s^2\, k_x(x_*, x_i)\,\alpha_i$, the static part.*

Part (b) says that the time kernel does more than down-weight stale evidence: it generates new escalations. Every region of the action space that stops receiving labels, directly or through its kernel neighbours, drifts back into the band on a fixed clock, whether or not the supervisor's tolerance has changed. With $\lambda = 90$ and $\tau_{\mathrm{high}} = 0.65$ ($z \approx 0.385$), an action with posterior mean $m = 2$ returns within $90 \ln(2/0.385) \approx 148$ steps. This forced re-escalation is an implicit re-audit, which is useful under drift, but it charges the supervisor for the static structure the model has already learned and it keeps the posterior close to the prior, which makes the gateway underconfident. The additive kernel keeps the static part and forgets only the tolerance and the interaction (**RQ3**);

the price is that it loses the implicit re-audit, which Section 3.7 restores explicitly. The Ornstein–Uhlenbeck process is also non-directional: it models change, not the one-way accumulation of trust. A sliding window of recent observations or online changepoint detection [1] are alternatives rather than equivalents.

3.6 Correlated generalization

A contextual learner that treats each (a, c) cell independently needs labels in every cell; the kernel transfers evidence between similar actions. With the featurization used here (reversibility, base sensitivity, blast radius and category of the tool; target sensitivity, destructive-argument flag and task of the context), approving `write_file` to `workspace_src` transfers to `write_file` to `workspace_tests`. Transfer cuts both ways: under the isotropic lengthscales of our experiments the prior correlation of `git_push` and `git_force_push` is 0.975 and that of `write_file` and `delete_file` on the same target 0.87, while moving target sensitivity from a sandbox to production infrastructure changes the context kernel only by a factor 0.71. Approval therefore also propagates toward dangerous neighbours. A new action inherits the prior, and with it an escalation, only if it is far from all labelled actions under the kernel; a new action that resembles approved actions inherits their approval. Per-feature lengthscales, a prior that makes f decrease in the risk attributes [44], or the epistemic margin of Remark 2 are the natural remedies; we measure the leak (Section 6.3) rather than fix it here.

3.7 Selective labels and audits

Only escalated actions are labelled. The GP posterior itself is not biased by this adaptive design when the model is correctly specified, since the decision to ask depends only on past observations; but every frequency computed from the labels is, including false-allow rates and any threshold tuning, and a confidently wrong ALLOW region is never revisited except through forgetting. We therefore add random audits: each auto-decided action is also shown to the human with probability ε.

Proposition 5 (Audits)**.** *(a) Let A_i be audit indicators with $\mathbb{E}[A_i] = \varepsilon > 0$ (independence is not required) and e_i the false-allow indicators of N auto-allowed actions. The Horvitz–Thompson estimator $\widehat{\mathrm{FA}} = \sum_i A_i e_i/(N\varepsilon)$ satisfies $\mathbb{E}\,\widehat{\mathrm{FA}} = \frac{1}{N}\sum_i e_i$. (b) If the true false-allow rate is at least $\alpha \in (0, 1]$, the probability that n independent audits are all clean is at most $(1-\alpha)^n \le e^{-\alpha n}$, which is at most δ once $n \ge \ln(1/\delta)/\alpha$.*

Part (b) is the familiar "rule of three" made exact: certifying a false-allow rate below 1% at 95% confidence takes 300 clean audits (**RQ4**). Published evaluations of deployed agent-action classifiers rest on samples of this order or smaller; for example, the false-negative rate reported for one production classifier on real dangerous actions is estimated from 52 examples [28].

4 Calibrating an Opaque LLM Judge

The gateway of Section 3 assumes a fixed taxonomy of tools whose risk attributes are known. When an agent routes its work through a general-purpose shell, an action is arbitrary code and no such taxonomy exists. This section asks what can be learned about the judge that then makes the allow/block decision, and how the gateway can sit on top of it (**RQ5**).

4.1 Judges in deployed agent harnesses

In Claude Code's auto mode, a transcript classifier reviews shell commands, web fetches, external tool calls and filesystem operations outside the project (read-only tools and in-project edits skip it); it "sees only user messages and the agent's tool calls", with the agent's own messages and tool outputs removed; it returns allow or block, a block comes back to the agent as a tool result telling it to find a safer path, and the human is asked only after three consecutive or twenty total blocks [28]. Its reported false-positive rate is 0.4% on 10,000 real actions and its false-negative rate 17% on 52 real overeager actions and 5.7% on 1,000 synthetic exfiltration attempts [28].

OpenAI's open-source Codex CLI makes different choices [39]. Its auto-review reviewer is a sub-agent consulted only when the static layers (sandbox, approval policy, command rules) would otherwise ask the user. It sees user and developer messages, project instruction files, the agent's messages, tool calls and tool outputs, but not the agent's reasoning; everything other than user and developer messages is marked as untrusted evidence, and the reviewer may run read-only commands to investigate. It returns allow or deny with an optional risk level, and its experimental asynchronous pre-scorer maps a single output token to a score of 0 or 1. A denial returns to the agent as a rejection, and three consecutive denials, or ten among the last fifty reviews, interrupt the turn.

The two designs sit at opposite ends of a trade-off in the judge's view: a reasoning-blind view limits what injected text can reach the judge, while a fuller view exposes more of the context that separates safe from unsafe actions, and more untrusted text with it. Neither exposes a calibrated score, and neither routes judge-allowed actions to a human. In both, the judge is a black box: only its verdict and, on some actions, the supervisor's decision are observable.

4.2 What can be known about a judge

Definition 2 (Judge reliability for a supervisor)**.** *For a judge J with verdicts in $\{\textsc{allow}, \textsc{block}\}$ and a supervisor with approval probability $q(x,t)$, the judge's false-allow and false-block rates over a period are $\mathrm{FA}_J = \Pr(y = 0 \mid J = \textsc{allow})$ and $\mathrm{FB}_J = \Pr(y = 1 \mid J = \textsc{block})$. The judge is (α, δ)-trustworthy for that supervisor and period if $\mathrm{FA}_J \le \alpha$ is certified with confidence $1 - \delta$. The reliability of an individual verdict is the posterior $\hat{p}\big(J(x), z(x), t\big) = \Pr(y = 1 \mid J(x), z(x), t, \mathcal{D})$ given the verdict, cheap observable features z and the labels so far.*

Definition 2 turns the two questions into two quantities: a period-level error rate (is the judge trustworthy?) and a per-action posterior (how likely is this verdict to be right?).

Proposition 6 (What cannot be learned without paired labels)**.** *(a) For every judge allow rate $a \in (0,1)$ and every $\varphi \in [0,1]$ there is a joint distribution of verdict and supervisor decision with $\Pr(J = \textsc{allow}) = a$ and $\mathrm{FA}_J = \varphi$. (b) Even the two marginals together do not identify FA_J: two joints with the same judge allow rate (0.8) and the same supervisor approval rate (0.9) have $\mathrm{FA}_J = 0$ and $\mathrm{FA}_J = 1/8$.*

A judge's error rate for a supervisor is therefore a property of the *joint* distribution, observable only through actions that carry both a verdict and a human label. Several judges do not remove the need: latent-class models become generically identifiable from three or more observers only if they are conditionally independent given the truth [2, 17], and the errors of language models are strongly correlated [23, 30].

Proposition 7 (Reliability is relational)**.** *Fix a judge, i.e. the set A of actions it allows, and a finite population with nonnegative weights and positive total weight on A. If supervisor q' is nowhere*

more permissive than supervisor q on A, the judge's false-allow rate for q' is at least its false-allow rate for q.

The same, unchanged judge is thus more or less trustworthy depending on whose decisions it stands in for and when: a certificate for one supervisor or one period does not transfer to another. This is where the judge and the gateway meet. The judge can estimate the static risk $r(x)$ of arbitrary code, which is what it is good at; the published analysis of the production classifier finds that in most misses it "correctly identifies the danger" but misjudges whether "a real consent signal in the session actually covers this action" [28], which is the supervisor-specific tolerance $\tau(t)$ of Definition 1.

Proposition 8 (Unbiased estimation with known propensities)**.** *If action i receives a human label with known probability $\pi_i > 0$ (for example $\pi_i = 1$ when it is escalated and $\pi_i = \varepsilon$ when it is an audited auto-decision), the Horvitz–Thompson sum $\sum_i L_i e_i/\pi_i$ is unbiased for $\sum_i e_i$.*

Proposition 8 generalizes Proposition 5(a) and lets escalations and audits both count, provided each label's propensity is logged; this is the logic of active and prediction-powered inference [4, 52]. In a design where the human sees an action only after repeated judge blocks, as in both deployed designs above, labels exist only for blocked actions, every propensity on judge-allowed actions is zero, and FA_J is not estimable at all.

4.3 A calibrated gateway over judge outputs

We therefore treat the judge's output as a feature rather than a decision (Figure 1). The gateway of Section 3 runs on $\big(J(x)$ or its score, a coarse command category $z(x)$, $t\big)$ with the additive kernel: its static part learns how this supervisor's approvals relate to the judge's output in each category, and its time part tracks the supervisor's tolerance. Its posterior $\hat{p}$ answers the per-verdict question; its Chow rule escalates rather than blocks where the judge's verdict is unreliable for this supervisor; and audits with logged propensities keep FA_J estimable. One limit is structural: the gateway cannot separate actions that look identical in its own features, so it cannot repair a judge blind spot its features do not expose. Argument-level features, for example static patterns in the command, are the remedy, not more calibration.

5 Experimental Setup

Why simulation. Empirically grounding the gateway requires proposed agent actions paired with human approve/deny judgments. R-Judge [50] supplies agent interaction records with human safe/unsafe labels and is a natural source for the static risk $r(x)$ of Definition 1. Agent-SafetyBench [51] and ToolEmu [45] widen coverage, but their labels come from automated judges rather than per-action human approval. No public dataset captures the longitudinal, per-supervisor signal the tolerance $\tau(t)$ requires, so we evaluate in a controlled simulation with a known oracle, the standard protocol for GP preference and level-set methods. Recovery of the oracle means that the inference is correct under the model, not that the world behaves like the model.

Environment. The action space has 18 tools with decision-time risk attributes, 8 target-sensitivity tiers and 7 task contexts. The oracle instantiates Definition 1: a static acceptability term linear in the risk attributes, a saturating trust term with an abrupt reset at $t = 750$ (the supervisor moves to an unfamiliar system), a task offset, and a three-way veto (irreversible *and* sensitive *and* low trust) that is active for roughly 90 steps after the reset; approvals are

Model	Regret	ASK %	Auto acc. %	False-allow %	ECE	Veto ALLOW
Per-tool Beta	0.660	0.0	67.0	1.7	0.314	15/109
Per-cell Beta	0.330	9.1	81.7	2.0	0.281	14/109
Product, λ=90	0.137	28.4	98.6	1.2	0.213	8/109
Product, evidence-selected	0.063	14.6	97.8	1.0	0.172	13/109
Linear probit + drift, ev.-sel.	0.059	10.0	96.7	2.0	0.062	31/109
Additive, λ=90	0.056	13.5	97.2	1.0	0.104	11/109
Additive, evidence-selected	0.059	11.9	96.8	1.5	0.093	20/109

Table 3: Learning a drifting supervisor under symmetric costs, no audits, scored phases, mean over 10 seeds. Regret is the cost-weighted headline; ASK, accuracy and false-allow (share of ALLOWs the oracle would deny) are its components. The last column pools, over seeds, the ALLOWs among the few actions for which the veto is active; read it as directional. The perfect-posterior escalation floor is 10.3%.

$y \sim \text{Bernoulli}(\Phi(f^*))$. The kernels see the risk attributes but never the trust state or the veto. Appendix C gives the full specification.

Protocol and metrics. Each run processes $N = 1500$ decisions prequentially; $[0, 560)$ is warm-up and the rest is scored, split into an early phase containing the reset, $[560, 1050)$, and a late phase, $[1050, 1500)$. Thresholds come from costs, never from labels: the symmetric band $(0.35, 0.65)$ and the safety-weighted band $(0.25, 0.90)$ of Section 3.4. The headline metric is *regret*: the mean Chow loss of the gateway's decisions minus that of the oracle policy that knows $\Phi(f^*)$, in units of one escalation. We also report the share of actions escalated (ASK), the accuracy of auto-decisions, the false-allow rate (share of ALLOWs the oracle would deny), the expected calibration error (ECE) of $\hat{p}$ against $\Phi(f^*)$, and the number of ALLOWs among the few actions for which the veto is active.

Models. We compare the product kernel ($\sigma^2 = 1.6$, $\lambda = 90$); the additive kernel, with a configuration fixed before any run ($\sigma_s^2 = 1.6$, $\sigma_g^2 = 1.0$, $\sigma_d^2 = 0.6$, $\lambda = 90$); evidence-selected variants of both; a Bayesian linear probit with the same shared drift term (the oracle's static term is linear in the features by construction, so this baseline is favoured); a per-cell Beta-Bernoulli learner (one estimate per tool, target and task); and a per-tool Beta-Bernoulli learner. Lengthscales are 1.1 (tool) and 1.2 (context) throughout. Tables use 10 seeds; the acquisition probe uses 20 paired seeds. All numbers in the paper are generated by the released code.

6 Learning a Drifting Supervisor

The first experiment asks what the choice of kernel, and in particular what it forgets, does to the human burden, to safety under drift, to the value of the ASK band as an acquisition rule, and to generalization to actions that are never labelled (**RQ1**, **RQ3**).

6.1 Kernel structure and human burden

The additive kernel halves the human burden. Table 3 gives the main comparison. Replacing the product kernel by the additive one cuts regret from 0.137 to 0.056, human labels from 28.4% to 13.5% of actions (a 7.4× rather than 3.5× reduction relative to escalating everything), and the calibration error from 0.213 to 0.104, with a false-allow rate of 1.0% against 1.2%. The price is auto-decision accuracy, which falls from 98.6% to 97.2%, because the additive gateway auto-decides

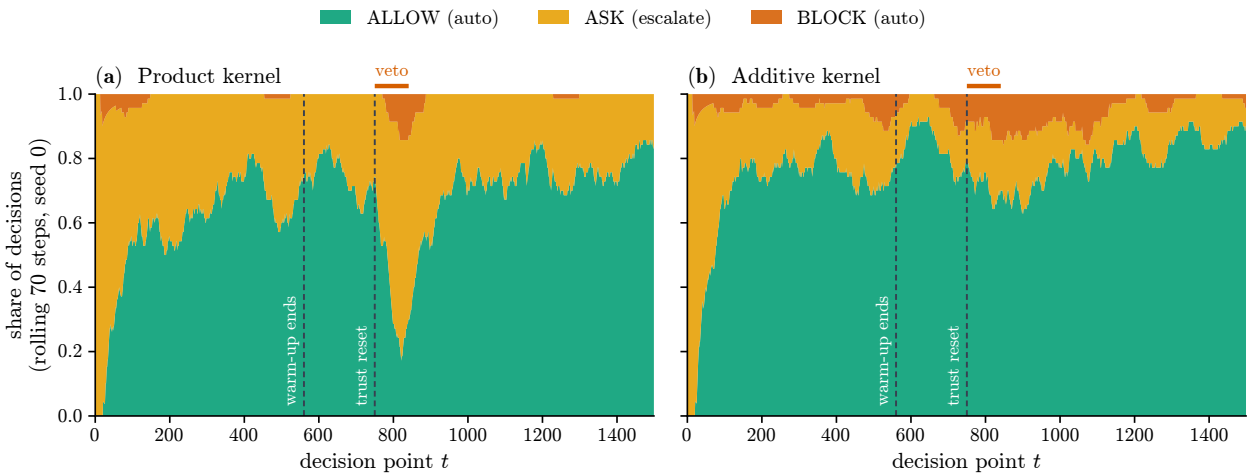


Figure 2: How the gateway's decisions evolve under symmetric costs (one seed; rolling shares over 70 decisions). (a) The product kernel escalates a large share of actions throughout, because every action it stops asking about decays back into the band; after the supervisor's trust reset it turns most of its allows into escalations. (b) The additive kernel escalates less once the static risk is learned, and after the reset it blocks, rather than asks about, many of the actions the veto makes unacceptable.

many more contestable actions. In the late phase the additive gateway escalates 11.5% of actions against a floor of 10.3%; the product gateway escalates 25.2%. Figure 2 shows where the difference comes from: the product gateway keeps returning actions to ASK long after the supervisor's tolerance has settled. Because the oracle is itself additive, the size of this advantage is indicative; what should transfer is the structural point that only the drifting components of a model should be forgotten. The simple baselines show why a structured model is needed at all: the per-tool learner auto-decides everything at 67.0% accuracy (regret 0.660), and the per-cell learner, which needs a label in every cell, reaches 81.7% (regret 0.330).

Forgetting makes a high-confidence bar unreachable. Under safety-weighted costs (Table 4) the product gateway escalates 99.1% of all actions (regret 0.509): its posterior never becomes confident enough to clear $\hat{p} > 0.9$, because every action it stops asking about decays toward $\hat{p} = \frac{1}{2}$ (Proposition 4). The additive gateway escalates 45.9% (regret 0.113) against a floor of 31.3%. Figure 3(a) isolates the mechanism: with the training set fixed, the product kernel's prediction for a benign `read_file` decays from 0.65 to 0.50 as the time since the last label grows, while the additive kernel's settles at its static part, 0.76. Panels (b) and (c) show the consequence under drift: both kernels follow the reset, but the product kernel tracks a fixed probe action from further below the truth and is the more underconfident of the two.

The linear baseline and the interaction. The Bayesian linear probit with the same shared drift is competitive on average metrics (regret 0.059, ECE 0.062), which is expected because the oracle's static term is linear in the features by construction. It cannot represent the veto interaction, and it allows 31/109 veto-window actions against 11/109 for the additive kernel. The kernel's structure (what is forgotten) thus matters more than its flexibility for average burden, while the non-parametric interaction term earns its keep on the rare, safety-relevant region; on real data the linear static term is unlikely to be correct.

Model	ASK %	Auto acc. %	Regret
Product, λ=90	99.1	99.2	0.509
Product, evidence-selected	55.3	99.9	0.171
Linear probit + drift, ev.-sel.	33.1	99.4	0.071
Additive, λ=90	45.9	99.7	0.113
Additive, evidence-selected	41.5	99.6	0.092

Table 4: Safety-weighted costs ($c_{\mathrm{FA}} = 10$, $c_{\mathrm{FB}} = 4$, $c_{\mathrm{ask}} = 1$; band $(0.25, 0.90)$), 10 seeds. Floor: 31.3%.

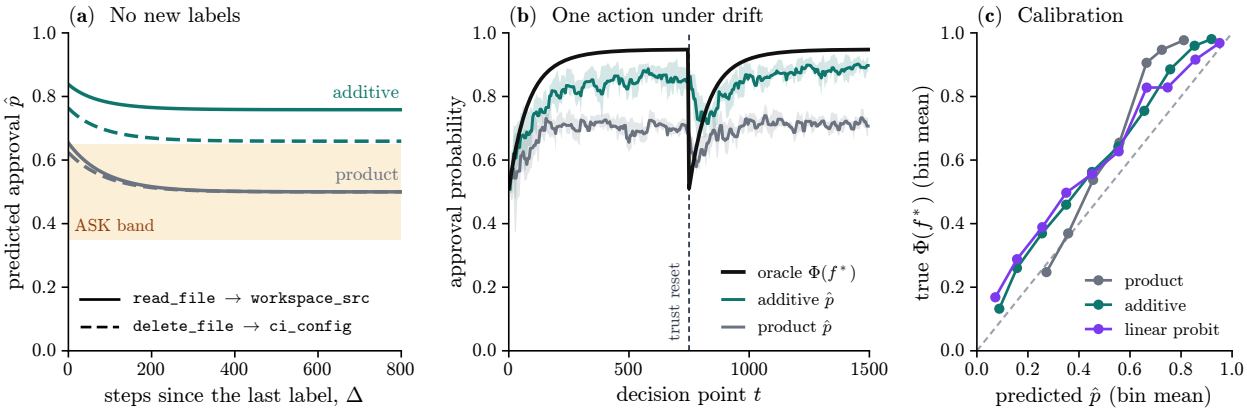


Figure 3: Forgetting and its cost. (a) Proposition 4 on a fixed training set: as the time since the last label grows, the product kernel returns every action to $\hat{p} = \frac{1}{2}$, inside the ASK band, while the additive kernel keeps the static part. (b) A fixed probe action under drift and the trust reset at $t = 750$: the oracle's approval probability and each kernel's $\hat{p}$ (mean and interquartile range over 10 seeds). (c) Reliability of $\hat{p}$ against the true approval probability, pooled over 10 seeds and the scored phases; points above the diagonal are underconfident.

Evidence selection optimizes average fit, not tail safety. On the labels the product-kernel gateway collected, the Laplace marginal likelihood ranks the product kernel last (mean log evidence -276.1 against -247.1 for the additive kernel, with the linear probit best in 8 of 10 seeds; this is evidence under a selected design, since only escalated actions are labelled). Selecting hyperparameters by evidence during the run chooses long memories (typically $\lambda = 480$ for the additive and 480–960 for the product kernel) and improves calibration, but it nearly doubles veto-window allows for the additive kernel (20/109 against 11/109). The marginal likelihood is an average-case criterion and the veto window holds about one percent of scored actions: it is a good criterion for structure and a poor one, on its own, for memory length. We therefore keep the pre-specified $\lambda = 90$.

Takeaway. Thresholds from stated costs work without any tuning labels (**RQ1**), and what the model forgets decides the human burden (**RQ3**): forgetting static risk roughly doubles it and makes a demanding confidence bar unreachable.

6.2 Is the ASK band a good acquisition rule?

Remark 1 separates escalation from acquisition: the band is the right place to ask for the action at hand, but not necessarily the right place to collect labels for the model. We compare, at a matched label budget and with 20 paired seeds, three query strategies: the band itself, uniform random

Kernel	Oracle	Band	Random	BALD	Band−random	BALD−random
Product, λ=90	stationary	82.0	82.4	81.2	-0.5 ± 0.7	-1.2 ± 0.8
Product, λ=90	changepoint	77.4	77.3	75.9	$+0.2 \pm 0.6$	-1.4 ± 1.0
Additive, λ=90	stationary	84.2	80.7	79.1	$+3.5 \pm 1.7$	-1.6 ± 2.0
Additive, λ=90	changepoint	77.6	74.5	76.0	$+3.1 \pm 1.2$	$+1.5 \pm 1.3$

Table 5: Is the ASK band a good acquisition rule? Prequential accuracy (%) on contestable actions ($\Phi(f^*) \in [0.15, 0.85]$) at a matched label budget; differences in percentage points ± paired standard error over 20 seeds.

querying, and a stream-based BALD sampler [27] that labels actions whose epistemic information is in the top fraction of a recent window. The outcome is prequential accuracy on contestable actions, those with $\Phi(f^*) \in [0.15, 0.85]$, where learning matters.

Results. With the product kernel the band is no better than random (Table 5; −0.5 and +0.2 percentage points, standard errors 0.7 and 0.6), on both a fully stationary oracle and the reset oracle, so non-stationarity is not the cause. With the additive kernel the same band beats random by +3.5 (±1.7) and +3.1 (±1.2) points. The BALD sampler does not improve on the band under either kernel. Two tempting explanations of the product kernel's failure do not survive. Class imbalance in the stream cannot be the cause, since both kernels see the same stream, and an information-theoretic rule is not the fix. Nor is it that the band spends its labels on intrinsically ambiguous actions: under the product kernel the band does place 27.1% of its labels on actions with true approval probability in [0.35, 0.65] (base rate 10.4%), at the same mean posterior variance as random querying (1.32 against 1.33), but under the additive kernel it places even more of its labels there and still beats random.

Takeaway. The controlling variable is forgetting (**RQ3**). Under the product kernel, actions return to the band on a fixed clock (Proposition 4), so band labels are spent relearning static risk the model has forgotten; once the kernel stops forgetting static risk, the escalation rule is also a good acquisition rule, and the fix is the kernel rather than the acquisition function.

6.3 Generalization to never-labelled actions

Section 3.6 predicts that the kernel transfers approval in both directions. We hold out two combinations and never label them: the benign `write_file` → `workspace_tests` and the dangerous `git_force_push` → `prod_infra`, and record the prequential decisions on their occurrences.

Results. On the benign combination the additive gateway auto-decides 96% of occurrences correctly by kernel transfer (product 81%, per-cell 0%, since it must ask about every unseen cell). On the dangerous combination, where the oracle denies 64 of the occurrences, every kernel model wrongly allows some: 7 (additive), 8 (product) and 8 (linear), against 5 for the per-tool learner, which has labels on the same tool at other targets, and none for the per-cell learner, which asks (Figure 4).

Takeaway. Correlated generalization is what makes the burden small, and it is also a leak: approval propagates to unseen dangerous neighbours. We report the leak rather than tune it away; the remedies of Section 3.6 and the audits of the next experiment address it.

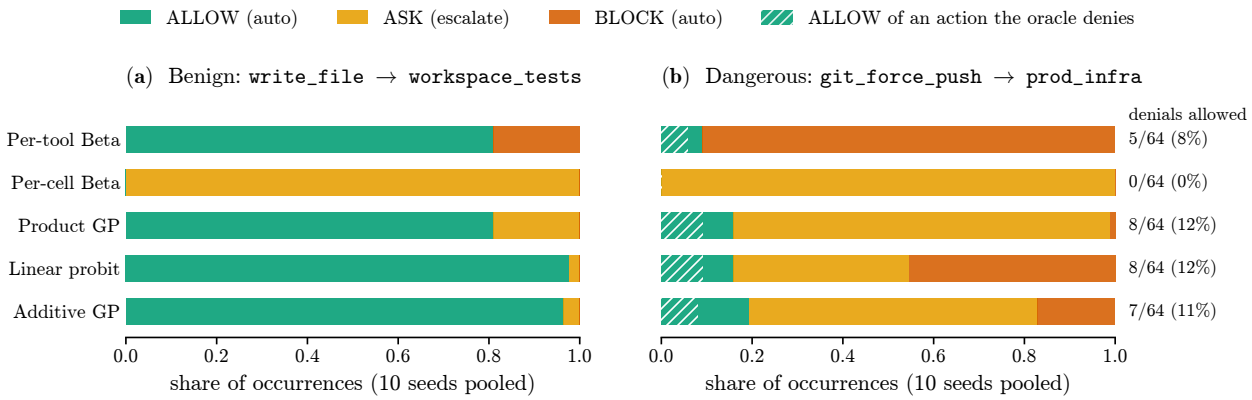


Figure 4: Decisions on two never-labelled combinations, pooled over 10 seeds (the linear probit is the evidence-selected linear model of Table 3). (a) Benign transfer: the kernel models auto-allow most occurrences correctly, while the per-cell learner must ask about every unseen cell. (b) A dangerous combination the oracle mostly denies: every kernel model auto-allows a share of the oracle's denials (hatched; counts at the right), because the combination resembles approved neighbours; only the per-cell learner, which asks every time, allows none.

Model	Labels/action %	Regret (audit billed)	Veto ALLOW	HT est./truth %	CI covers
Product, λ=90	28.4 → 30.2	0.137 → 0.119 (0.157)	8/109 → 7/109	5.9 / 6.1	100%
Linear probit + drift, ev.-sel.	10.0 → 15.7	0.059 → 0.043 (0.088)	31/109 → 14/109	6.3 / 6.8	100%
Additive, λ=90	13.5 → 18.2	0.056 → 0.046 (0.089)	11/109 → 5/109	7.2 / 6.4	100%
Additive, evidence-selected	11.9 → 16.2	0.059 → 0.054 (0.098)	20/109 → 20/109	10.0 / 7.4	90%

Table 6: Auditing auto-decisions: random audits of $\varepsilon = 5\%$ of auto-decided actions, symmetric costs, 10 seeds; arrows show no audits → audits. Regret charges decisions only, so audits are free in it; the value in parentheses bills each audit as one escalation. The last columns compare the Horvitz–Thompson estimate of the realized false-allow rate with its true value, and give the share of seeds whose Jeffreys 95% interval on the audited fraction covers the truth.

7 Auditing Auto-Decisions

The second experiment asks what random audits cost and what they buy (**RQ4**). We add audits at $\varepsilon = 5\%$ of auto-decided actions under symmetric costs and compare the Horvitz–Thompson estimate of the realized false-allow rate with its true value.

Results. For the additive gateway audits raise labels per action from 13.5% to 18.2% and halve veto-window allows (11/109 → 5/109; Table 6); for the linear baseline they cut veto-window allows from 31/109 to 14/109. Regret as defined charges decisions only, so an audit of an auto-decided action is free in it; on that measure the additive gateway's regret falls from 0.056 to 0.046. Billing each audit as one escalation, it rises to 0.089 instead: audits pay for themselves in average regret only if one costs at most 0.22 of an escalation, which a post-hoc review may or may not. The Horvitz–Thompson estimate of the realized false-allow rate is 7.2% against a true 6.4%, and a Jeffreys interval on the audited fraction covers the truth in 100% of seeds. With about forty audits per seed the estimate is noisy, as Proposition 5(b) predicts.

	Symmetric costs				Safety costs	
Policy	**Regret**	**Labels %**	**FA %**	**Blind FA**	**Regret**	**Labels %**
Judge only (allow/block)	0.296	0.0	0.9	28/29	0.565	0.0
Judge, escalate instead of block	0.088	25.3	0.9	28/29	0.173	25.3
Always ask	0.720	100.0	–	0/29	0.516	100.0
Calibrated gateway	0.086	11.8	2.2	23/29	0.150	46.7
Calibrated gateway + 5% audits	0.075	16.5	2.3	26/29	0.168	52.4
Calibrated (product kernel) + audits	0.146	27.2	2.6	17/29	0.462	93.7

Table 7: Calibrating a simulated judge, verdict input, 10 seeds. FA is the share of ALLOWs the oracle would deny; Blind FA pools, over seeds, the blind-spot actions the oracle would deny that were auto-allowed. Audits are free in regret (Section 7).

Takeaway. Audits make the gateway's error rate estimable, not small, and they improve safety inside the veto window; once their cost is counted, they do not lower average regret (**RQ4**).

8 Calibrating a Simulated LLM Judge

The third experiment replaces the tool taxonomy by an opaque judge (**RQ5**). It asks whether a calibrated gateway on top of the judge improves on the judge's own decisions, whether the judge's reliability for the supervisor moves while the judge does not, and whether audits can track it. Every setting was fixed in a pre-registration committed before the first run, and the study ran without deviations.

Design. We keep the oracle, stream and seeds of Section 5 and add a judge. The judge sees the action's static risk but not the supervisor's trust state, task preferences or veto; it blocks when a noisy score of the risk exceeds a fixed threshold, and it has a blind spot: destructive shell commands look benign to it. The gateway sees only the judge's verdict (or score), an eight-way command category and time. We compare six policies: the judge alone; escalating the judge's blocks instead of enforcing them; asking about every action; the calibrated gateway of Section 4.3 with and without audits; and the same gateway with the product kernel. This simulated judge is much stricter than the production classifier: it blocks about a quarter of all actions, most of which this supervisor would approve, against a reported false-positive rate of 0.4%. Gains from turning its blocks into escalations are therefore dominated by avoided false blocks and reflect the pre-registered threshold rather than a prediction about any deployed system; the relational and estimation results do not depend on it.

Does calibration improve on the judge? Letting the judge decide alone gives regret 0.296 with no human labels (Table 7); escalating its blocks instead of enforcing them gives 0.088 at 25.3% labels. The calibrated gateway reaches the same regret (0.086) with half the labels (11.8%), and lower regret with the judge's score (0.068) or with audits (0.075, audits unbilled). It is also more permissive than the judge (false-allow 2.2% against 0.9%), because it learns that this supervisor approves much of what the judge blocks: the gateway estimates whether *this supervisor* would approve, not whether the action is safe, and a judge's fixed policy can be safer than a supervisor's revealed tolerance. Under safety-weighted costs the sign flips (false-allow 0.1%) at 46.7% labels, and calibration buys regret 0.150 against 0.173 for escalating the judge's blocks, at about twenty more labels per hundred actions. With the product kernel the gateway again escalates almost

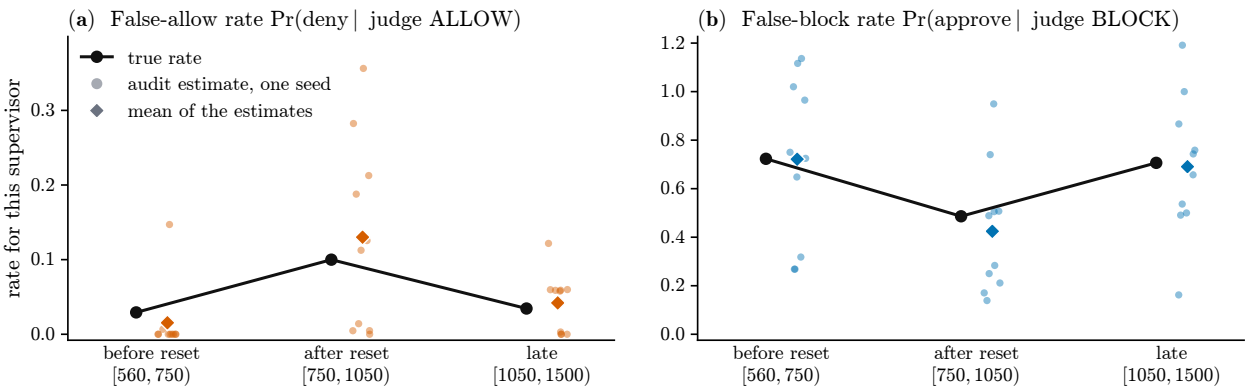


Figure 5: The same, unchanged judge's error rates for one supervisor, in three windows around the supervisor's trust reset. (a) False-allow rate Pr(deny | J = ALLOW); (b) false-block rate Pr(approve | J = BLOCK). Lines: true rates; dots: per-seed Horvitz–Thompson estimates from escalations and audits; diamonds: their mean. The false-allow rate more than triples after the reset although the judge is unchanged. The per-seed estimates scatter widely because each window holds few labels; the estimator is unbiased but not bounded, so single estimates can exceed one.

everything under safety costs (93.7% labels), the forgetting collapse of Proposition 4 outside the tool taxonomy.

Does the judge's reliability move? The judge's reliability for this supervisor moves while the judge does not (Figure 5): its false-allow rate is 2.9% before the supervisor's trust reset, 10.0% after it and 3.4% late, as Proposition 7 predicts. Horvitz–Thompson estimates from the gateway's labels track these values (1.5%, 13.0% and 4.2% on average), unbiased but noisy within a window, with standard deviations across seeds of 4.4 to 12.1 points, because each window holds only 8 to 24 human labels on judge-allowed actions, escalations and audits together. Without audits they are undefined in every seed.

Can calibration catch a blind spot? No. Under symmetric costs 23/29 of the blind-spot actions this supervisor would deny are still allowed, because they are indistinguishable from benign shell commands in the gateway's features; under safety costs the count falls to 3/29 only because the gateway escalates judge-allowed shell commands as a class.

Takeaway. A calibrated gateway uses the judge's verdict as evidence and spends human labels where the verdict is unreliable for this supervisor; the judge's trustworthiness is a moving, relational quantity that only audited labels can measure; and calibration cannot see past the features it is given.

9 Discussion

9.1 Answers to the research questions

RQ1: Decision rule. The three-tier gateway is Chow's reject option. Its thresholds follow from what a false allow, a false block and an interruption cost, and need no labels; its only input is the

mean approval probability; and its escalation rate cannot fall below the supervisor's own ambiguity at those costs (10.3% and 31.3% of actions in our two cost settings).

RQ2: Feedback. Unary approve/deny feedback identifies the allow/deny boundary; pairwise comparisons identify only a ranking. The gateway is GP classification with a reject option and a level-set objective, a relative of preferential Bayesian optimization but not an instance of it.

RQ3: Memory. A model of a drifting supervisor should forget the supervisor's tolerance, not the risk of the actions. Forgetting both, as a separable kernel does, re-escalates every action on a fixed clock, roughly doubles the human burden, makes a demanding confidence bar unreachable and destroys the value of the ASK band as an acquisition rule. An additive kernel that forgets only what drifts avoids all four; correlated generalization remains a leak toward dangerous neighbours.

RQ4: Evaluation. Because the gateway labels only what it escalates, its error rate is invisible without audits. Random audits with logged propensities make it estimable without bias, but not precisely: certifying a false-allow rate below α takes $\ln(1/\delta)/\alpha$ clean audits, and audits do not lower average regret once they are billed.

RQ5: Opaque judges. Whether a judge is trustworthy is a statistical claim about a moving target. It cannot be read off the judge's verdicts (Proposition 6); it changes with the supervisor (Proposition 7); and certifying it takes clean audits in proportion to $1/\alpha$ (Proposition 5(b)): 12 for $\alpha = 25\%$, 300 for $\alpha = 1\%$, while the quantity drifts on the supervisor's timescale. Certification and tracking pull against each other, and the honest form of a trust claim is "false-allow rate below α for this supervisor over the last n audits". How likely a given verdict is to be right is the calibrated posterior $\hat{p}$, which needs the same labels and cannot exceed what the features can distinguish.

9.2 Implications for agent harnesses

Four design choices follow for a harness that gates agent actions. First, uncertainty should escalate rather than refuse: in our judge study, turning the judge's blocks into escalations cut regret from 0.296 to 0.088. Second, thresholds should be derived from stated costs, which also documents the operator's risk appetite, rather than tuned on labels the gateway does not have. Third, a model of the supervisor should keep what it has learned about static risk and forget only the supervisor's tolerance. Fourth, a small fraction of auto-decisions, including judge-allowed ones, should be audited with the sampling propensity logged next to each label; both deployed designs of Section 4.1 show the human only blocked actions, so neither can estimate its judge's false-allow rate for a given user. The judge is then best used as a feature of a calibrated gateway, with argument-level features for the blind spots calibration cannot reach. Each judge call is itself a paid model call, so a cascade that consults the judge only when the gateway is uncertain, in the spirit of selective evaluation with escalation [29], is a natural extension. An implementation for agent harnesses needs these choices: both kernels, cost-derived thresholds, audits whose sampling propensities are logged with each label, a certification check, and a featurizer over a judge's verdict and a command category.

9.3 Limitations

Simulation. All results come from a controlled simulation whose oracle follows Definition 1. The oracle is additive by construction, which favours the additive kernel and, through its linear static

term, the linear baseline; the size of the gains is indicative, not a prediction. The simulated judge is far stricter than the production classifiers it stands in for, so gains from escalating its blocks are inflated. The main remaining gap is empirical: a longitudinal study of real supervisors approving real agent actions.

The human as ground truth. Our supervisor is noisy but unbiased. Real supervisors are not: users approve 97% of permission prompts in one deployment, and in a controlled study human reviewers caught 13.6% of dangerous commands against 89% for the classifier, with their block rate falling from about 17% early in a session to about 5% after fifty or more prompts [41], consistent with habituation to repeated warnings [3]. A gateway calibrated to a rubber-stamping supervisor learns to rubber-stamp; a fatigue-aware model would make the cost of asking, and the noise of the answer, grow with the escalation rate, which argues for fewer and better-targeted escalations.

Adversaries. Judges are attack surfaces: agent-written prompt injections obtained arbitrary shell execution past two production classifiers in 79% of trials [43], and reliability estimated from benign audits says nothing about adaptive attacks. Control protocols with trusted monitors and audit budgets address that setting [6, 26].

Modelling choices. The decision rule is one-step (Remark 1); a rule that prices the value of information might ask less. The Laplace approximation is known to be over-cautious, the Ornstein–Uhlenbeck time kernel is non-directional, and the isotropic lengthscales let approval leak to dangerous neighbours; the forgetting rate is pre-specified, because evidence selection prefers memories that are too long for the rare veto region. Reliability estimates within a window rest on a few dozen labels.

Scope of the formal verification. The machine-checked proofs (Appendix B) keep the link function abstract (strictly increasing, or continuous at zero with value one half), which covers the probit and logistic links alike. They do not formalize the Laplace approximation: Proposition 4 is proved for any predictor that is linear in the cross-covariance vector with coefficients fixed by the training data, which is the form of the Laplace mean and of exact GP regression. Proposition 2(a) is proved for an arbitrary probability measure over the latent value, and Propositions 3 and 7 for finite action populations.

10 Concluding Remarks

Deciding when an agent may act alone is not a threshold to hand-tune but a latent tolerance to learn, and the decision on top of it is a reject option whose thresholds follow from what errors and interruptions cost. Three structural facts shape what such a gateway can achieve: unary feedback is what identifies the allow/deny boundary; a model that forgets everything charges the supervisor again for what it already knew and cannot reach a demanding confidence bar; and the labels the gateway collects are selected, so audits are needed to know its error rate. In simulation, a kernel that separates static risk from drifting tolerance halves the human burden and turns the ASK band into an effective acquisition rule, while correlated generalization still leaks approval to unseen dangerous actions. When the tool taxonomy is replaced by an opaque judge over arbitrary shell actions, the same machinery calibrates the judge to a supervisor, but the judge's trustworthiness becomes a moving, relational quantity that only audited labels can measure, and calibration cannot see past

the features it is given. What remains is to measure real supervisors over time, including how their own reliability as ground truth declines as they are asked more often.

References

[1] Ryan Prescott Adams and David J. C. MacKay. Bayesian online changepoint detection. *arXiv preprint arXiv:0710.3742*, 2007. URL https://arxiv.org/abs/0710.3742.

[2] Elizabeth S. Allman, Catherine Matias, and John A. Rhodes. Identifiability of parameters in latent structure models with many observed variables. *The Annals of Statistics*, 37(6A): 3099–3132, 2009. doi: 10.1214/09-AOS689.

[3] Bonnie Brinton Anderson, C. Brock Kirwan, Jeffrey L. Jenkins, David Eargle, Seth Howard, and Anthony Vance. How polymorphic warnings reduce habituation in the brain: Insights from an fMRI study. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI)*, pages 2883–2892, 2015. doi: 10.1145/2702123.2702322.

[4] Anastasios N. Angelopoulos, Stephen Bates, Clara Fannjiang, Michael I. Jordan, and Tijana Zrnic. Prediction-powered inference. *Science*, 382(6671):669–674, 2023. doi: 10.1126/science.adi6000.

[5] Alessio Benavoli and Dario Azzimonti. A tutorial on learning from preferences and choices with Gaussian processes. *Foundations and Trends in Machine Learning*, 19(1):1–120, 2026. doi: 10.1108/ftmal-03-2025-0124.

[6] Aryan Bhatt, Cody Rushing, Adam Kaufman, Tyler Tracy, Vasil Georgiev, David Matolcsi, Akbir Khan, and Buck Shlegeris. Ctrl-Z: Controlling AI agents via resampling, 2025.

[7] Ilija Bogunovic, Jonathan Scarlett, and Volkan Cevher. Time-varying Gaussian process bandit optimization. In *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 51 of *PMLR*, pages 314–323, 2016. URL https://proceedings.mlr.press/v51/bogunovic16.html.

[8] Ilija Bogunovic, Jonathan Scarlett, Andreas Krause, and Volkan Cevher. Truncated variance reduction: A unified approach to Bayesian optimization and level-set estimation. In *Advances in Neural Information Processing Systems*, volume 29, 2016. URL https://proceedings.neurips.cc/paper_files/paper/2016/hash/ce78d1da254c0843eb23951ae077ff5f-Abstract.html.

[9] Eric Brochu, Vlad M. Cora, and Nando de Freitas. A tutorial on Bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning. *arXiv preprint arXiv:1012.2599*, 2010. URL https://arxiv.org/abs/1012.2599.

[10] Alan Chan, Rebecca Salganik, Alva Markelius, Chris Pang, Nitarshan Rajkumar, Dmitrii Krasheninnikov, Lauro Langosco, Zhonghao He, Yawen Duan, Micah Carroll, Michelle Lin, Alex Mayhew, Katherine Collins, Maryam Molamohammadi, John Burden, Wanru Zhao, Shalaleh Rismani, Konstantinos Voudouris, Umang Bhatt, Adrian Weller, David Krueger, and Tegan Maharaj. Harms from increasingly agentic algorithmic systems. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pages 651–666. ACM, 2023. doi: 10.1145/3593013.3594033. URL https://doi.org/10.1145/3593013.3594033.

[11] Alan Chan, Carson Ezell, Max Kaufmann, Kevin Wei, Lewis Hammond, Herbie Bradley, Emma Bluemke, Nitarshan Rajkumar, David Krueger, Noam Kolt, Lennart Heim, and Markus Anderljung. Visibility into AI agents. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pages 958–973. ACM, 2024. doi: 10.1145/3630106.3658948. URL https://doi.org/10.1145/3630106.3658948.

[12] C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970. doi: 10.1109/TIT.1970.1054406.

[13] Wei Chu and Zoubin Ghahramani. Preference learning with Gaussian processes. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 137–144. ACM Press, 2005. doi: 10.1145/1102351.1102369. URL https://doi.org/10.1145/1102351.1102369.

[14] David Cohn, Les Atlas, and Richard Ladner. Improving generalization with active learning. *Machine Learning*, 15(2):201–221, 1994. doi: 10.1007/BF00993277.

[15] Sanjoy Dasgupta. Two faces of active learning. *Theoretical Computer Science*, 412(19):1767–1781, 2011. doi: 10.1016/j.tcs.2010.12.054.

[16] Sanjoy Dasgupta and Daniel Hsu. Hierarchical sampling for active learning. In *Proceedings of the 25th International Conference on Machine Learning (ICML)*, pages 208–215, 2008. doi: 10.1145/1390156.1390183.

[17] A. P. Dawid and A. M. Skene. Maximum likelihood estimation of observer error-rates using the EM algorithm. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 28 (1):20–28, 1979. doi: 10.2307/2346806.

[18] Ewart J. de Visser, Marieke M. M. Peeters, Malte F. Jung, Spencer Kohn, Tyler H. Shaw, Richard Pak, and Mark A. Neerincx. Towards a theory of longitudinal trust calibration in human–robot teams. *International Journal of Social Robotics*, 12(2):459–478, 2020. doi: 10.1007/s12369-019-00596-x. URL https://doi.org/10.1007/s12369-019-00596-x.

[19] David K. Duvenaud, Hannes Nickisch, and Carl Edward Rasmussen. Additive Gaussian processes. In *Advances in Neural Information Processing Systems*, volume 24, 2011. URL https://proceedings.neurips.cc/paper_files/paper/2011/hash/4c5bde74a8f110656874902f07378009-Abstract.html.

[20] Ran El-Yaniv and Yair Wiener. On the foundations of noise-free selective classification. *Journal of Machine Learning Research*, 11:1605–1641, 2010. URL https://jmlr.org/papers/v11/el-yaniv10a.html.

[21] Aditya Gangrade, Anil Kag, Ashok Cutkosky, and Venkatesh Saligrama. Online selective classification with limited feedback. In *Advances in Neural Information Processing Systems*, volume 34, pages 14529–14541, 2021.

[22] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html.

[23] Shashwat Goel, Joschka Strüber, Ilze Amanda Auzina, Karuna K. Chandra, Ponnurangam Kumaraguru, Douwe Kiela, Ameya Prabhu, Matthias Bethge, and Jonas Geiping. Great models think alike and this undermines AI oversight. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *PMLR*, pages 19621–19678, 2025.

[24] Javier González, Zhenwen Dai, Andreas Damianou, and Neil D. Lawrence. Preferential Bayesian optimization. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1282–1291. PMLR, 2017. URL https://proceedings.mlr.press/v70/gonzalez17a.html.

[25] Alkis Gotovos, Nathalie Casati, Gregory Hitz, and Andreas Krause. Active learning for level set estimation. In *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1344–1350, 2013. URL https://www.ijcai.org/Proceedings/13/Papers/202.pdf.

[26] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, pages 16295–16336, 2024.

[27] Neil Houlsby, Ferenc Huszár, Zoubin Ghahramani, and Máté Lengyel. Bayesian active learning for classification and preference learning. *arXiv preprint arXiv:1112.5745*, 2011. URL https://arxiv.org/abs/1112.5745.

[28] John Hughes. How we built Claude Code auto mode: A safer way to skip permissions. Anthropic Engineering Blog, March 2026. URL https://www.anthropic.com/engineering/claude-code-auto-mode.

[29] Jaehun Jung, Faeze Brahman, and Yejin Choi. Trust or escalate: LLM judges with provable guarantees for human agreement. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2407.18370.

[30] Elliot Myunghoon Kim, Avi Garg, Kenny Peng, and Nikhil Garg. Correlated errors in large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *PMLR*, pages 30038–30066, 2025.

[31] Malte Kuss and Carl Edward Rasmussen. Assessing approximate inference for binary Gaussian process classification. *Journal of Machine Learning Research*, 6:1679–1704, 2005. URL https://www.jmlr.org/papers/volume6/kuss05a/kuss05a.pdf.

[32] Himabindu Lakkaraju, Jon Kleinberg, Jure Leskovec, Jens Ludwig, and Sendhil Mullainathan. The selective labels problem: Evaluating algorithmic predictions in the presence of unobservables. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 275–284, 2017. doi: 10.1145/3097983.3098066.

[33] John D. Lee and Katrina A. See. Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1):50–80, 2004. URL https://doi.org/10.1518/hfes.46.1.50_30392.

[34] Benjamin Letham, Phillip Guan, Chase Tymms, Eytan Bakshy, and Michael Shvartsman. Look-ahead acquisition functions for Bernoulli level set estimation. In *Proceedings of the 25th International Conference on Artificial Intelligence and Statistics*, volume 151 of *PMLR*, pages 8493–8513, 2022.

[35] David Madras, Toniann Pitassi, and Richard Zemel. Predict responsibly: Improving fairness and accuracy by learning to defer. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/hash/09d37c08f7b129e96277388757530c72-Abstract.html.

[36] Thomas P. Minka. Expectation propagation for approximate Bayesian inference. In *Proceedings of the Seventeenth Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 362–369. Morgan Kaufmann, 2001. URL https://tminka.github.io/papers/ep/minka-ep-uai.pdf.

[37] Meredith Ringel Morris, Jascha Sohl-Dickstein, Noah Fiedel, Tris Warkentin, Allan Dafoe, Aleksandra Faust, Clement Farabet, and Shane Legg. Position: Levels of AGI for operationalizing progress on the path to AGI. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 36308–36321. PMLR, 2024. URL https://proceedings.mlr.press/v235/morris24b.html.

[38] Hussein Mozannar and David Sontag. Consistent estimators for learning to defer to an expert. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *PMLR*, pages 7076–7087, 2020.

[39] OpenAI. Codex CLI. Open-source repository, commit ea647275560ffbcd4cf240d048eab44aa66ba9ae, 2026. URL https://github.com/openai/codex.

[40] Christopher J. Paciorek and Mark J. Schervish. Nonstationary covariance functions for Gaussian process regression. In *Advances in Neural Information Processing Systems 16 (NIPS)*. MIT Press, 2003. URL https://proceedings.neurips.cc/paper/2003/hash/326a8c055c0d04f5b06544665d8bb3ea-Abstract.html.

[41] Conner Phillippi. Auto mode is now the default in Claude Code for Pro, Max, and Team plans. Claude Blog, August 2026. URL https://claude.com/blog/auto-mode-default-in-claude-code.

[42] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, Cambridge, MA, 2006. ISBN 978-0-262-18253-9. doi: 10.7551/mitpress/3206.001.0001. URL https://gaussianprocess.org/gpml/chapters/RW.pdf.

[43] Alex Remedios, Simon Storf, Fabien Roger, and John Hughes. Red-teaming auto mode: Improving blocking classifiers against malign coding agents, 2026. URL https://arxiv.org/abs/2609.19587.

[44] Jaakko Riihimäki and Aki Vehtari. Gaussian processes with monotonicity information. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics*, volume 9 of *PMLR*, pages 645–652, 2010.

[45] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=GEcwtMk1uA. Spotlight.

[46] Burr Settles. Active learning literature survey. Computer Sciences Technical Report 1648, University of Wisconsin–Madison, 2009. URL https://research.cs.wisc.edu/techreports/2009/TR1648.pdf.

[47] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016. doi: 10.1109/JPROC.2015.2494218. URL https://doi.org/10.1109/JPROC.2015.2494218.

[48] Yonadav Shavit, Sandhini Agarwal, Miles Brundage, Steven Adler, Cullen O'Keefe, Rosie Campbell, Teddy Lee, Pamela Mishkin, Tyna Eloundou, Alan Hickey, Katarina Slama, Lama Ahmad, Paul McMillan, Alex Beutel, Alexandre Passos, and David G. Robinson. Practices for governing agentic AI systems. White paper, OpenAI, 2023. URL https://cdn.openai.com/papers/practices-for-governing-agentic-ai-systems.pdf.

[49] Yanan Sui, Alkis Gotovos, Joel Burdick, and Andreas Krause. Safe exploration for optimization with Gaussian processes. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *PMLR*, pages 997–1005, 2015.

[50] Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. R-judge: Benchmarking safety risk awareness for LLM agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1467–1490. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-emnlp.79. URL https://aclanthology.org/2024.findings-emnlp.79/.

[51] Zhexin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. Agent-safetybench: Evaluating the safety of LLM agents, 2024. URL https://arxiv.org/abs/2412.14470.

[52] Tijana Zrnic and Emmanuel Candès. Active statistical inference. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, pages 62993–63010, 2024. URL https://proceedings.mlr.press/v235/zrnic24a.html.

A Proofs

Proposition 1. (a) Φ is injective, so $\Phi(f(x)) = \Phi(g(x))$ gives $f(x) = g(x)$ for every x. (b) $(f(x) + c) - (f(x') + c) = f(x) - f(x')$. Given x, take $g = f + c$ with $c = -f(x) - 1$ if $f(x) > 0$ and $c = -f(x) + 1$ otherwise; then g has the same pairwise likelihoods and $g(x) = \mp 1$.

Proposition 2. (a) Each loss is affine in p, so by linearity of the integral $\int L_{\text{ALLOW}}(\Phi)\, d\mu = (1 - \int \Phi\, d\mu)\, c_{\text{FA}}$ and similarly for BLOCK. (b) Clear denominators in $c_{\text{ask}}/c_{\text{FB}} < 1 - c_{\text{ask}}/c_{\text{FA}}$. (c) ALLOW beats ASK iff $p > \tau_{\text{high}}$ and BLOCK beats ASK iff $p < \tau_{\text{low}}$; ALLOW beats BLOCK iff $p > c_{\text{FA}}/(c_{\text{FA}}+c_{\text{FB}})$, and when the band is non-empty this point lies strictly inside it, so the pairwise comparisons compose to rule (7). (d) $\min(L_{\text{ALLOW}}, L_{\text{BLOCK}})$ is at most the convex combination with weights $c_{\text{FB}}/(c_{\text{FA}} + c_{\text{FB}})$ and $c_{\text{FA}}/(c_{\text{FA}} + c_{\text{FB}})$, which equals $c_{\text{FA}}c_{\text{FB}}/(c_{\text{FA}} + c_{\text{FB}}) \le c_{\text{ask}}$ when the band is empty.

Proposition 3. Inside the open band ASK is strictly better than both auto-decisions (Proposition 2(c)), so a pointwise-optimal policy cannot choose either; the band is therefore a subset of the escalated set, and nonnegative weights give the inequality.

Proposition 4. For $s \le t$ and $\Delta \ge 0$, $|t+\Delta-s| = \Delta+|t-s|$, so $k_{\text{time}}(t+\Delta, s) = e^{-\Delta/\lambda} k_{\text{time}}(t, s)$. Every term of the product-kernel predictor carries one such factor, giving (a) for the mean, and the quadratic form carries two, giving $e^{-2\Delta/\lambda}$. For (b), $\sqrt{1+\sigma^2} \ge 1$, so $|\mu/\sqrt{1+\sigma^2}| \le e^{-\Delta/\lambda}|m| \to 0$ and continuity gives $\hat{p} \to \frac{1}{2}$, which eventually lies in any open band around $\frac{1}{2}$. For the explicit time, $\Delta \ge \lambda \ln(m/z)$ gives $e^{-\Delta/\lambda} m \le z$, and monotonicity of Φ gives $\hat{p} \le \Phi(z) = \tau_{\text{high}}$. (c) Only the second and third components of k_{add} carry the time factor.

Proposition 5. (a) By linearity, $\mathbb{E}\sum_i A_i e_i = \varepsilon \sum_i e_i$. (b) If the rate is at least α, each audit is clean with probability at most $1-\alpha$, and $1-\alpha \le e^{-\alpha}$; then $e^{-\alpha n} \le \delta$ iff $n \ge \ln(1/\delta)/\alpha$.

Proposition 6. Write the joint of verdict and decision as the four cell probabilities (allow & deny, allow & approve, block & deny, block & approve). (a) Take $(a\varphi,\ a(1-\varphi),\ 1-a,\ 0)$: the judge allows with probability a and $\text{FA}_J = a\varphi/a = \varphi$. (b) Take $(0, 0.8, 0.1, 0.1)$ and $(0.1, 0.7, 0, 0.2)$: both have judge allow rate 0.8 and supervisor approval rate 0.9, and FA_J is 0 and $0.1/0.8 = 1/8$.

Proposition 7. On A, $1-q' \ge 1-q$ pointwise; summing with the nonnegative weights over A and dividing by the positive weight of A (the same for both supervisors) preserves the inequality.

Proposition 8. By linearity, $\mathbb{E}\sum_i L_i e_i/\pi_i = \sum_i \pi_i e_i/\pi_i = \sum_i e_i$.

B Machine-Checked Proofs

The decision-theoretic and algebraic cores of all eight propositions are formalized in Lean 4 (v4.34.1) with Mathlib (v4.34.1). The development has no unproved goals (`sorry`), and every theorem depends only on Lean's standard axioms (`propext`, `Classical.choice`, `Quot.sound`). Table 8 maps each proposition to its theorems; Section 9.3 states the scope of the formalization. The simulation code checks the same statements numerically in its test suite.

Result	Module	Theorems
Prop. 1	`Identifiability`	`unary_identifies`, `unary_identifies_boundary`, `pairwise_shift_invariant`, `pairwise_cannot_identify_threshold`
Prop. 2	`Chow`	`expectedLoss_allow`, `expectedLoss_block`, `band_nonempty_iff`, `allow/block/ask_strictly_optimal`, `decide_optimal`, `ask_never_needed`
Prop. 3	`Floor`	`optimal_asks_on_band`, `escalation_floor`
Prop. 4	`Forgetting`	`kT_shift`, `meanProd_shift`, `quadForm_shift`, `exp_decay_tendsto`, `eventually_ask`, `reentry_bound`, `reentry_prob`, `meanAdd_shift`, `meanAdd_tendsto`
Prop. 5	`Audit`	`audit_unbiased`, `one_sub_pow_le_exp`, `certify_sample_size`
Props. 6–8	`Judge`	`judge_unidentified`, `judge_marginals_not_enough`, `judgeFA_mono`, `ht_unbiased`
Remark 2	`Identifiability`	`lcb_iff`

Table 8: Propositions and their machine-checked counterparts (31 theorems).

C Simulation Details

Actions and features. Each of the 18 tools has a category (read, search, version control, execution, write, database, network, deploy), a reversibility class (reversible, hard to reverse, irreversible), a base sensitivity in $[0, 1]$, a blast radius from 0 (one file) to 3 (external), and a probability that a destructive argument accompanies it. Targets have sensitivities from 0 (`sandbox_tmp`) to 1 (`prod_infra`). At each step a task is drawn uniformly, a tool category from task-specific weights, a tool uniformly within the category, a target uniformly among those plausible for the category, and the destructive-argument flag from the tool's probability. The tool feature vector is (reversibility/2, base sensitivity, blast radius/3, category one-hot); the context feature vector is (target sensitivity, destructive-argument flag, task one-hot).

Oracle. The latent tolerance is $f^*(x, t) = a_0 - w\, r(x) + \rho\, \mathrm{trust}(t) + o(\mathrm{task}) - v\, \mathbb{1}[\mathrm{veto}]$ with $a_0 = 1.4$, $w = 3.4$, $\rho = 1$ and $v = 3$. The risk is $r(x) = 0.28\,\mathrm{rev}/2 + 0.32\,\mathrm{target\ sensitivity} + 0.12\,\mathrm{blast}/3 + 0.16\,\mathrm{destructive\ argument} + 0.12\,\mathrm{base\ sensitivity}$. Trust accumulates as $\mathrm{trust}(t) = \theta\,(1 - e^{-(t-t_0)/\kappa})$ with $\theta = 1.6$, $\kappa = 130$, $t_0 = 0$ before the reset and $t_0 = 750$ after it. Task offsets range from -0.25 (data migration) to $+0.15$ (security hardening). The veto is active when the tool is irreversible, the target sensitivity is at least 0.65 and trust is below $\theta/2$, that is, for about $\kappa \ln 2 \approx 90$ steps after each (re)start. The stationary oracle of the acquisition probe saturates trust immediately and never resets it, so the veto is never active.

Models. The GP models refit every 8 steps. Evidence-selected models re-select by Laplace marginal likelihood every 64 labels, over $\sigma^2 \in \{1.6, 3.2\}$ and $\lambda \in \{60, 120, 240, 480, 960\}$ for the product kernel; over the splits $(\sigma_s^2, \sigma_g^2, \sigma_d^2) \in \{(1.6, 1.0, 0.6), (3.2, 1.0, 0.6), (1.6, 2.0, 0.6), (1.6, 1.0, 1.6)\}$ and $\lambda \in \{60, 120, 240, 480\}$ for the additive kernel; and over $\lambda \in \{60, 120, 240, 480\}$ for the linear probit. The Beta-Bernoulli baselines start from $\mathrm{Beta}(1, 1)$.

Acquisition probe. For each of 20 seeds, the band strategy runs first and fixes the label budget; random querying labels each action with the band's overall query rate, and the BALD sampler labels an action when its BALD score is in the top fraction, equal to that rate, of the last 100 scores (using the band for the first 20 steps). All three share the stream.

Judge. The judge's score is $s = \mathrm{logistic}\big(8\,(r' - 0.5) + \eta\big)$ with $\eta \sim \mathcal{N}(0, 1)$, where r' is the action's risk except for destructive execution commands, which the judge perceives as $r' = 0.2$ (the blind spot); it blocks when $s > 0.5$. The gateway's tool features are the judge's verdict (or score) and the eight-way category one-hot; its context features are constant. Reliability windows are $[560, 750)$, $[750, 1050)$ and $[1050, 1500)$.